ANEXO TÉCNICO

Bem Viver ILPI — Arquitetura do banco de dados

Documento complementar para avaliação técnica • PostgreSQL 16 • Row-Level Security • revisão de 11/10/2026

30	75	18	17	49
tabelas, todas com RLS	políticas	funções de segurança	gatilhos de auditoria	chaves estrangeiras

1. Princípio de projeto

A autorização é imposta na camada de dados, não na aplicação. A API conecta ao PostgreSQL sob um papel de baixo privilégio (`app_user`) e, a cada requisição, injeta a identidade do usuário autenticado em uma variável de sessão transacional. Todas as políticas de **Row-Level Security (RLS)** derivam a visibilidade de cada linha a partir dessa identidade. A aplicação não escreve nenhuma cláusula `WHERE` de permissão.

> **Consequência de segurança:** uma falha na camada de aplicação (bug, rota mal protegida, injeção) não expõe dados de outra ILPI ou de outro perfil, porque o filtro é aplicado pelo próprio SGBD. É defesa em profundidade com o ponto de decisão na origem do dado.

2. Injeção de identidade (aplicação → banco)

A API abre uma transação por requisição e define a identidade com `set_config(..., true)`, cujo escopo é local à transação — equivalente a `SET LOCAL`. Isso é essencial com *pool* de conexões: a identidade nunca vaza de uma requisição para a próxima.

api/server.js

```
async function comUsuario(idUsuario, fn) {
  const client = await pool.connect();
  try {
    await client.query("BEGIN");
    // Identidade + fuso das ILPIs (horários de medicação são hora local de PE).
    // Ambos com escopo da transação: funciona com pool e com PgBouncer (Neon).
    await client.query("SELECT set_config('app.id_usuario', $1, true), set_config('TimeZone', $2, true)",
      [String(idUsuario), FUSO]);
    const resultado = await fn(client);
    await client.query("COMMIT");
    return resultado;
  } catch (erro) {
    await client.query("ROLLBACK");
    throw erro;
  } finally {
    client.release();
  }
}
```

No banco, a identidade é lida por uma função `STABLE`, com tratamento seguro de valor ausente:

rls_ilpi.sql

```
CREATE OR REPLACE FUNCTION fn_usuario_atual() RETURNS int
LANGUAGE sql STABLE AS
$$ SELECT NULLIF(current_setting('app.id_usuario', true), '')::int $$;
```

3. Autenticação dentro do banco

A senha é guardada como hash bcrypt (`pgcrypto`) e conferida pelo próprio PostgreSQL. A aplicação **não tem permissão de ler a coluna** `senha_hash`: o login chama uma função `SECURITY DEFINER`, que roda como dono da tabela e devolve apenas quem é o usuário.

api/setup_api.sql

```
CREATE OR REPLACE FUNCTION fn_autenticar(p_email text, p_senha text)
RETURNS TABLE (id_usuario int, perfil perfil_usuario, funcao funcao_profissional)
LANGUAGE sql STABLE SECURITY DEFINER SET search_path = public, pg_temp AS $$
    SELECT u.id_usuario, u.perfil, p.funcao
      FROM usuario u
      LEFT JOIN profissional p ON p.id_usuario = u.id_usuario
     WHERE u.email = lower(trim(p_email))   -- e-mails são guardados em minúsculas
       AND u.ativo
       AND u.senha_hash = crypt(p_senha, u.senha_hash)
       AND (p.id_profissional IS NULL OR p.ativo)
$$;
```

api/server.js — rota POST /auth/login

```
const { rows } = await pool.query(
  "SELECT id_usuario, perfil, funcao FROM fn_autenticar($1, $2)",
  [email, senha]
```

O privilégio sobre a tabela de contas é concedido coluna a coluna:

rls_ilpi.sql

```
REVOKE SELECT, UPDATE ON usuario FROM app_user;
GRANT SELECT (id_usuario, email, perfil, ativo, criado_em) ON usuario TO app_user;
GRANT UPDATE (ativo) ON usuario TO app_user;
```

4. Papel de conexão e privilégios

rls_ilpi.sql — concessão de base

```
GRANT USAGE ON SCHEMA public TO app_user;
GRANT SELECT, INSERT, UPDATE ON ALL TABLES IN SCHEMA public TO app_user;
GRANT USAGE ON ALL SEQUENCES IN SCHEMA public TO app_user;
```

A concessão de base não inclui `DELETE`: exclusões físicas são a exceção, e desativações são lógicas (colunas `ativo`), preservando histórico clínico e auditoria. A partir dela, o script **estreita** os privilégios tabela a tabela:

Tabela	Ajuste	Motivo
`usuario`	SELECT só em 5 colunas; UPDATE só em `ativo`	hash da senha fora do alcance da aplicação
`pet_virtual`	UPDATE só em nome, espécie, acessório e cor	pontos só mudam pela `fn_pontuar_pet` (seção 6); o nível é coluna gerada
`evento_pet`, `log_auditoria`	sem INSERT nem UPDATE	escritos apenas por função e gatilho
`ilpi`	sem INSERT nem UPDATE	cadastro de instituição é administrativo
`recado_mural`, `restricao_alimentar`	**DELETE concedido**	únicas exclusões físicas (abaixo)

As duas exceções ao “sem DELETE”. O recado do mural é conteúdo do familiar que o escreveu (a LGPD garante corrigir e eliminar o próprio conteúdo); a restrição alimentar é configuração do cuidado, não registro de evento. Em ambas, quem pode apagar é decidido pelo RLS e cada exclusão fica na trilha de auditoria:

rls_ilpi.sql

```
CREATE POLICY del_recado_autor ON recado_mural FOR DELETE TO app_user
    USING (id_responsavel = fn_meu_id_responsavel());
GRANT UPDATE (texto, editado_em), DELETE ON recado_mural TO app_user;
```

5. Funções de segurança e políticas RLS

Funções auxiliares (SECURITY DEFINER, STABLE)

São `SECURITY DEFINER` para poderem consultar tabelas de cadastro sem dar ao `app_user` acesso direto a elas. Cada uma responde a uma pergunta de autorização. Todas as 17 funções que rodam como dono fixam `search_path = public, pg_temp`: ninguém consegue “trocar” uma tabela por um objeto temporário de mesmo nome — e um bloco de conferência no fim do script aborta a instalação se alguma ficar sem. As duas últimas separam o **vínculo** (cadastro, Xodó, mural, atividades) do **prontuário**, que a família só vê com consentimento LGPD vigente:

rls_ilpi.sql

```
CREATE OR REPLACE FUNCTION fn_minha_ilpi_profissional() RETURNS int
LANGUAGE sql STABLE SECURITY DEFINER SET search_path = public, pg_temp AS
$$ SELECT id_ilpi FROM profissional WHERE id_usuario = fn_usuario_atual() $$;

CREATE OR REPLACE FUNCTION fn_sou_psicologo() RETURNS boolean
LANGUAGE sql STABLE SECURITY DEFINER SET search_path = public, pg_temp AS
$$ SELECT EXISTS (SELECT 1 FROM profissional
                  WHERE id_usuario = fn_usuario_atual()
                    AND funcao = 'psicologo') $$;

CREATE OR REPLACE FUNCTION fn_idosos_sob_minha_responsabilidade() RETURNS SETOF int
LANGUAGE sql STABLE SECURITY DEFINER SET search_path = public, pg_temp AS
$$ SELECT v.id_idoso
     FROM vinculo_responsavel v
     JOIN responsavel r USING (id_responsavel)
    WHERE r.id_usuario = fn_usuario_atual() $$;

CREATE OR REPLACE FUNCTION fn_idosos_com_consentimento_saude() RETURNS SETOF int
LANGUAGE sql STABLE SECURITY DEFINER SET search_path = public, pg_temp AS
$$ SELECT v.id_idoso
     FROM vinculo_responsavel v
     JOIN responsavel r USING (id_responsavel)
    WHERE r.id_usuario = fn_usuario_atual()
      AND EXISTS (SELECT 1 FROM consentimento_lgpd c
                  WHERE c.id_idoso = v.id_idoso
                    AND c.categoria = 'dados_saude'
                    AND c.data_revogacao IS NULL) $$;
```

Exemplo de política: leitura multi-perfil

A tabela `registro_bem_estar` é lida por três perfis distintos, cada um com sua regra, numa única política de `SELECT`:

rls_ilpi.sql

```
ALTER TABLE registro_bem_estar ENABLE ROW LEVEL SECURITY;
ALTER TABLE registro_bem_estar FORCE ROW LEVEL SECURITY;

CREATE POLICY sel_bem_estar ON registro_bem_estar FOR SELECT TO app_user
    USING (
        fn_ilpi_do_idoso(id_idoso) = fn_minha_ilpi_profissional()
        OR id_idoso IN (SELECT fn_idosos_com_consentimento_saude())
        OR id_idoso = fn_meu_id_idoso()
    );
```

Exemplo de política: escrita com WITH CHECK

Na escrita, `WITH CHECK` impede que um perfil grave em nome indevido — o idoso só registra para si; a equipe, apenas para idosos da própria ILPI:

rls_ilpi.sql

```
CREATE POLICY ins_bem_estar_self ON registro_bem_estar FOR INSERT TO app_user
    WITH CHECK (id_idoso = fn_meu_id_idoso() AND id_profissional IS NULL);

CREATE POLICY ins_bem_estar_equipe ON registro_bem_estar FOR INSERT TO app_user
    WITH CHECK (
        id_profissional = fn_meu_id_profissional()
        AND fn_ilpi_do_idoso(id_idoso) = fn_minha_ilpi_profissional()
    );
```

Cobertura total: tabelas de apoio herdam o recorte

Todas as 30 tabelas têm RLS ligado com `FORCE`. Nas tabelas filhas, a política não repete a regra: um `EXISTS` sobre a tabela-mãe já passa pelo RLS dela, e o recorte é herdado. Assim, os itens de uma avaliação ficam visíveis exatamente para quem vê a avaliação.

rls_ilpi.sql

```
CREATE POLICY sel_item_avaliacao ON item_avaliacao FOR SELECT TO app_user
    USING (EXISTS (SELECT 1 FROM avaliacao a WHERE a.id_avaliacao = item_avaliacao.id_avaliacao));

CREATE POLICY sel_evento_pet ON evento_pet FOR SELECT TO app_user
    USING (EXISTS (SELECT 1 FROM pet_virtual p WHERE p.id_pet = evento_pet.id_pet));
```

> `FORCE ROW LEVEL SECURITY` garante que o RLS se aplique inclusive ao dono das tabelas, não só ao `app_user` — fechando a brecha de o proprietário do schema ignorar as políticas.

6. Regra de negócio no banco: os pontos do pet virtual

Cada check-in, presença em atividade ou partida rende pontos ao pet do idoso (o “Xodó”), e cada 100 pontos sobem um nível. Antes, cada rota da API inseria o evento e atualizava o pet por conta própria — e, como só o dono pode alterar o próprio pet, os pontos de check-ins feitos pela equipe e os do adversário no dominó eram descartados em silêncio pelo RLS, e o nível nunca subia. Agora existe **uma única porta de entrada**, `fn_pontuar_pet`, que confere a permissão, grava o evento e soma os pontos na mesma transação. O nível não é gravado por ninguém: é uma coluna gerada (`GENERATED ALWAYS AS (1 + pontos / 100) STORED`), que nunca discorda dos pontos.

rls_ilpi.sql — trecho de fn_pontuar_pet

```
IF NOT COALESCE(p_id_idoso = v_eu
        OR v_ilpi = fn_minha_ilpi_profissional()
        OR (p_origem = 'partida_domino' AND EXISTS (
                SELECT 1 FROM partida_domino pd
                 WHERE pd.id_jogador1 = v_eu AND pd.id_jogador2 = p_id_idoso
                   AND pd.data_hora >= now() - INTERVAL '1 minute')), false) THEN
    RAISE EXCEPTION 'Sem permissão para pontuar o pet deste idoso.' USING ERRCODE = '42501';
END IF;

SELECT * INTO v_pet FROM pet_virtual WHERE id_idoso = p_id_idoso FOR UPDATE;
IF NOT FOUND THEN RETURN; END IF;   -- idoso sem pet: nada a fazer

INSERT INTO evento_pet (id_pet, origem, pontos_ganhos) VALUES (v_pet.id_pet, p_origem, p_pontos);
UPDATE pet_virtual SET pontos = v_pet.pontos + p_pontos WHERE id_pet = v_pet.id_pet
RETURNING pet_virtual.nivel INTO v_nivel;            -- recalculado pela coluna gerada
```

api/server.js — chamada explícita a partir da rota de check-in

```
const { rows: ganho } = await c.query(
  "SELECT pet, pontos, nivel, subiu_nivel FROM fn_pontuar_pet($1, 'checkin_bem_estar', 10)", [id_idoso]);
```

> **Armadilha do NULL:** para quem não é profissional, `fn_minha_ilpi_profissional()` é NULL. Sem o `COALESCE`, a condição inteira vira NULL, `IF NOT NULL` não dispara a exceção, e qualquer idoso poderia pontuar o pet de outro. O teste automatizado da seção 9 pegou esse caso.

7. Modelo de dados e integridade

Domínios controlados (ENUM)

15 tipos enumerados garantem vocabulário fechado e alinhamento a normas. Exemplos:

Tipo	Valores	Origem
`grau_dependencia`	I, II, III	RDC 502/2021
`funcao_profissional`	cuidador, tecnico_enfermagem, enfermeiro, medico, psicologo, fisioterapeuta, nutricionista, assistente_social, gestor	equipe da ILPI
`instrumento_avaliacao`	KATZ, LAWTON, GDS15, WHOQOL_OLD, ICOPE	escalas geriátricas e OMS
`prioridade_plantao`	rotina, atencao, urgente	passagem de turno
`acao_auditoria`	leitura, criacao, edicao, exclusao	trilha LGPD

Ações referenciais

Ação	Onde	Por quê
NO ACTION (padrão)	quase todas as FKs para `idoso` e `profissional`	apagar um idoso com histórico clínico é bloqueado de propósito
ON DELETE CASCADE	`item_avaliacao`, `evento_pet`, `evolucao_meta`, `ciencia_plantao`	dependência de existência: não existem sem o registro principal
ON DELETE SET NULL	`idoso.id_usuario`, `registro_bem_estar.id_profissional`, `recado_plantao.id_idoso`	o registro sobrevive; só perde a referência opcional
ON UPDATE CASCADE	`vinculo_responsavel`, `participacao`	associativas: acompanham a renumeração de um cadastro (ex.: fusão de duplicados)

Integridade de domínio

O banco não confia no formato que chega da aplicação:

Regra	Como
CPF e CNPJ com dígitos verificadores	funções `IMMUTABLE` (`fn_cpf_valido`, `fn_cnpj_valido`) dentro de `CHECK`; recusam também 111.111.111-11
e-mail único sem diferenciar maiúsculas	`CHECK` de formato em minúsculas; a API e o login normalizam
telefone e UF	só dígitos com DDD; sigla de uma das 27 UFs
nível do pet	coluna gerada a partir dos pontos
um consentimento vigente por categoria	índice único parcial (abaixo)

schema_ilpi.sql

```
CREATE FUNCTION fn_cpf_valido(p text) RETURNS boolean
LANGUAGE plpgsql IMMUTABLE STRICT PARALLEL SAFE AS $$
    ...
CONSTRAINT ck_idoso_cpf CHECK (fn_cpf_valido(cpf))
nivel    INT NOT NULL GENERATED ALWAYS AS (1 + pontos / 100) STORED,
```

Índices

O PostgreSQL indexa sozinho a PK e o UNIQUE, mas não a coluna que referencia. Todas as 49 chaves estrangeiras têm índice (82 índices no total): sem isso, apagar ou renumerar a linha referenciada — justamente o que `ON DELETE SET NULL` e `ON UPDATE CASCADE` fazem — varreria a tabela filha inteira, e as funções do RLS, que fazem JOIN pelas FKs, também. Índices parciais (`WHERE ativo`) só servem à FK quando o filtro é `IS NOT NULL`; por isso `idoso(id_ilpi)` e `prescricao(id_idoso)` ganharam índice completo.

Auditoria, consentimento e views

Escritas em tabelas sensíveis disparam `fn_trigger_auditoria()`, que grava autor, ação, tabela e registro em `log_auditoria` — no nível do banco, seja qual for o caminho da escrita. O `consentimento_lgpd` não sofre exclusão: concessão e revogação recebem carimbo temporal, preservando o histórico exigido pela LGPD (Art. 8º §5º). E ele **tem efeito**: as políticas das tabelas clínicas usam `fn_idosos_com_consentimento_saude()`, então revogar o consentimento de dados de saúde esconde o prontuário da família na hora, sem mudar uma linha da API. Só pode haver um vigente por categoria:

schema_ilpi.sql

```
CREATE UNIQUE INDEX ux_consentimento_vigente
    ON consentimento_lgpd (id_idoso, categoria)
    WHERE data_revogacao IS NULL;
```

As views usam `security_invoker = true` (PostgreSQL 15+), respeitando o RLS de quem consulta, e não o do criador — evitando o vazamento clássico por view.

8. Fluxo do dado (pipeline transacional)

Todo dado percorre um caminho único — não existe rota de escrita alternativa que contorne validação, identidade ou auditoria:

Estágio	O que acontece	Garantia
1. Entrada	o app monta o objeto e envia via `api()`; sem conexão, check-ins e partidas de jogos entram na fila offline e seguem o mesmo caminho depois	nenhum registro se perde por queda de rede
2. Validação	a rota valida formato e faixas (humor 1–5, dor 0–10, jogo e resultado válidos); o banco repete com CHECK e ENUM	dado malformado não toca o banco
3. Identidade + transação	transação por requisição; `set_config('app.id_usuario', ..., true)`	atomicidade; identidade não vaza entre requisições
4. Persistência	SQL explícito e parametrizado (`$1, $2…`); RLS autoriza linha a linha; FKs, ENUMs, gatilhos de auditoria e `fn_pontuar_pet` aplicam-se	autorização, integridade e trilha na origem
5. Leituras derivadas	painel, prontuário unificado, resumo do dia, relatório PDF e as 8 consultas analíticas do gestor leem das mesmas tabelas, sob o mesmo RLS	fonte única da verdade, sem duplicação

Não há pipeline analítico (ETL/warehouse) — decisão deliberada de escopo: as consultas diretas indexadas atendem o volume atual. Como os dados nascem validados, tipados e com relações íntegras, um estágio analítico pode ser acoplado futuramente sem retrabalho.

9. Como foi verificado

- **Instalação limpa** a cada mudança: schema, carga de dados, RLS e preparo da API rodam do zero sem erro. A própria carga aborta se alguma tabela ficar com menos de 30 linhas.
- **Regressão de leitura:** 76 rotas da API × 7 perfis (idoso, responsável, enfermeira, psicólogo, cuidador, enfermeira de outra ILPI e gestor) fotografadas antes e depois de ligar o RLS nas tabelas restantes, e de novo depois das restrições de domínio e dos índices — **532 combinações, nenhuma diferença** de status ou de quantidade de linhas.
- **Escritas e permissões:** 87 verificações automatizadas — por exemplo, a enfermeira de outra ILPI não lê hashes, contas, itens de avaliação, eventos do pet nem partidas alheias; a idosa não insere evento de pet direto nem pontua o pet de outro; o gestor cadastra pessoas só na própria ILPI; CPF com dígito errado, telefone curto e e-mail sem @ voltam como erro 400; revogar o consentimento zera o prontuário da família (bem-estar, sinais, avaliações, metas, nutrição) sem tirar o cartão, o Xodó, o mural nem o acesso da equipe, e concedê-lo de novo devolve tudo.
- **No navegador** (Playwright): CRUD do mural e das restrições, as 8 consultas com gráfico, e partidas reais de Cuidar do Xodó e Buraco gravando pontos.

10. Pontos que convidam à discussão

Trade-offs assumidos e questões em aberto para avaliação:

- **Custo do RLS:** políticas com subconsultas (`fn_*` , `EXISTS` herdados) executam por linha/consulta. As FKs usadas nas funções já são indexadas; para volumes maiores, vale um `EXPLAIN ANALYZE` nas consultas quentes e medir o custo das chamadas `SECURITY DEFINER` .
- **Cache de identidade:** as funções são `STABLE` (reavaliadas por consulta). Há espaço para memoizar dentro da transação se o perfil for consultado muitas vezes.
- **Regras de negócio no banco:** `fn_pontuar_pet` centraliza a pontuação no SGBD. Ganha-se consistência (nenhum caminho escapa da regra) e perde-se portabilidade e facilidade de teste unitário — onde traçar essa linha?
- **Horários da prescrição em `TIME[]` :** escolha deliberada do PostgreSQL — os horários só existem dentro da prescrição e a consulta de adesão os expande com `unnest` . Uma tabela `horario_prescricao` seria a forma 1FN estrita; discutir se o ganho compensa.
- **`registro_saude` × tabelas específicas:** a tabela genérica de anotações ficou ao lado de `sinal_vital` e `intercorrencia` , criadas depois com colunas próprias. Hoje serve a anotações livres e consultas; consolidar ou manter?
- **Modelagem usuario/idoso/responsavel:** separação por papel (em vez de herança) — discutir se atende bem quando uma pessoa acumula papéis (ex.: funcionário que também é responsável).
- **Multi-tenancy:** hoje o isolamento entre ILPIs é por RLS num schema único. Para escala, comparar com schema-por-tenant ou particionamento.
- **Exclusão física restrita:** só duas tabelas aceitam DELETE. A desativação lógica preserva histórico, mas cresce indefinidamente — discutir política de retenção/arquivamento à luz da LGPD (minimização vs. rastreabilidade).

Resumo: o projeto aplica autorização declarativa no SGBD (RLS em todas as tabelas + funções + `FORCE`), autenticação sem expor o hash à aplicação, integridade por FKs, CHECKs e ENUMs, e conformidade por auditoria e consentimento versionado. As questões acima são os pontos onde a opinião de um especialista agregaria mais valor.